K.S. INSTITUTE OF TECHNOLOGY, BANGALORE

Department of Computer Science & Design

BCSL657D – DevOps Engineering Lab

Program 10

Continuous Integration with Azure DevOps

Subject Code	BCSL657D
Subject Title	DevOps Engineering Lab
Program No.	10
Semester / Scheme	VI | VTU CBCS 2021
Total Marks	20
Duration	2 Hours

Prepared by: Prof. Raghavendra S | CS&D Department | April 2026

1. Demystifying Azure DevOps – Read This First!

⚠ **Important:** Many students feel Azure DevOps is complex and impossible to run. This section will show you it is just 5 simple phases, each with clear steps and expected screenshots. You CANNOT go wrong if you follow this manual.

1.1 What is Azure DevOps?

Azure DevOps is Microsoft's cloud platform for automating software development workflows. Think of it as a set of online services that work together:

Service	What it does	Analogy
Azure Repos	Stores your source code in Git	*Like GitHub – your code lives here in the cloud*
Azure Pipelines	Automates build, test, and deploy	*Like a robot that runs mvn package every time you push code*
Agent	A program on YOUR machine that runs pipeline jobs	*Like a delivery person who picks up tasks from the cloud and runs them locally*
YAML Pipeline	Configuration file that defines what the robot does	*Like a recipe – tells the robot exactly what steps to follow*

1.2 The Big Picture – 5 Phases to Complete This Lab

Here is the complete picture of what you will build today. Each phase takes about 15–20 minutes.

Phase	Name	What you do	Result
1	**Project Setup**	Create Azure DevOps org + project, push code to Azure Repos	☑ Code visible at dev.azure.com
2	**Agent Setup**	Install and register self-hosted agent on your lab PC	☑ Agent shows 'Online' in the portal
3	**Pipeline YAML**	Create azure-pipelines.yml defining Maven build steps	☑ Pipeline definition saved in the repo
4	**First Run**	Run pipeline, grant one-time agent pool permission	☑ BUILD SUCCESS – green checkmark

5	**CI Auto-Trigger**	Edit code, git push, watch pipeline fire automatically	☑ New run appears without clicking anything

1.3 Pre-requisites Checklist

ⓘ **Note:** Verify every item below before starting. Items marked ★ are configured by your lab instructor before the session.

#	Requirement	Version	Who sets up
1	Java JDK installed (verify: java -version)	JDK 21	Student
2	Apache Maven installed (verify: mvn -version)	3.9.x	Student
3	Git installed and configured	2.x+	Student
4	Azure DevOps account at dev.azure.com	—	★ Instructor
5	Azure DevOps Organization created (ksit-devops)	—	★ Instructor
6	Maven Java project (student-app) created locally	—	Student
7	Self-hosted agent installed on lab PC	v4.272+	★ Instructor

2. Step-by-Step Procedure

PHASE 1 | Project Setup – Azure DevOps Project & Code Repository

Step 1 – Create Azure DevOps Project

1. Open a browser and go to https://dev.azure.com and sign in with your Microsoft account.
2. Click + New Project (top-right). Enter the project name: student-app-devops. Set Visibility to Private. Click Create.
3. You will land on the project overview page with sections: Boards, Repos, Pipelines, etc.

> 💡 **Tip:** If your instructor has already created the organization, you will be added as a member. Just navigate to dev.azure.com/{org-name} to see it.

Step 2 – Push Your Maven Project to Azure Repos

1. Inside your project, click Repos → Files. Copy the HTTPS clone URL shown on screen.
2. Open Command Prompt on your machine, navigate to your student-app folder, and run:

```
cd D:\Exercise\student-app
git init
git add .
git commit -m "Initial commit – student Maven app"
git remote add origin <PASTE-YOUR-AZURE-REPO-URL-HERE>
git push -u origin main
```

> ☑ **Expected Output:** Git should print: Writing objects: 100%... To https://dev.azure.com/... main -> main

> ℹ **Note:** If git push asks for a username/password: use your Azure DevOps username and a Personal Access Token (PAT) as the password. Your instructor will provide this.

PHASE 2 | Self-Hosted Agent – Connecting Your Machine to Azure DevOps

What is a Self-Hosted Agent and why do we need it?

When Azure DevOps needs to run a build, it needs a machine to run it on. A self-hosted agent is a lightweight program you install on your own machine (the lab PC) that:

- Constantly listens to Azure DevOps for new build jobs
- Picks up a job when your pipeline is triggered
- Runs the build using tools already installed on your machine (JDK, Maven, Git)
- Sends the results (logs, pass/fail) back to Azure DevOps

> ⓘ **Note:** Think of it like a Swiggy delivery partner – the agent registers itself on the platform, waits for an order (build job), picks it up, executes it locally, and reports back.

Step 3 – Configure the Self-Hosted Agent

1. In Azure DevOps, go to Project Settings (bottom-left gear icon) → Agent Pools → Default pool.
2. Click New Agent. Download the Windows agent ZIP file. Extract it to D:\azure-agent.
3. Open PowerShell as Administrator and run the configuration:

```
cd D:\azure-agent
.\config.cmd

# When prompted, enter:
# Server URL  :  https://dev.azure.com/ksit-devops
# Auth type   :  PAT
# PAT value   :  <your Personal Access Token>
# Agent pool  :  Default
# Agent name  :  RAGHU165-agent   (use your machine name)
# Work folder :  Press Enter for default (_work)
```

4. Start the agent so it can receive jobs:

```
.\run.cmd
```

5. Go to Project Settings → Agent Pools → Default → Agents tab. Verify your agent is listed as Online (green dot).

☑ **Expected Output:** Agent name appears in the list with a green Online indicator. Note the agent name and machine name – you will see these in the build logs later.

PHASE 3 | Pipeline YAML – Defining the Build as Code

What is azure-pipelines.yml?

Instead of clicking buttons in a GUI to configure a build, Azure DevOps lets you define the entire pipeline as a YAML file stored inside your repository. This means:

- The pipeline config is version-controlled just like your code
- Any commit to the repo can automatically trigger a build
- Every team member can see and modify the pipeline config

Step 4 – Create the Pipeline

1. In Azure DevOps, go to Pipelines → Create Pipeline.
2. Select Azure Repos Git as the code location.
3. Select your repository: student-app-devops.
4. Choose Starter Pipeline. Replace ALL content in the editor with the YAML below:

```yaml
# azure-pipelines.yml
# This file defines the CI pipeline for the student-app Maven project

trigger:
  - main                    # Run pipeline on every push to 'main' branch

pool:
  name: Default             # Use the self-hosted agent in the Default pool

steps:
  - checkout: self          # Step 1: Download the repository code

  - task: Maven@3           # Step 2: Run Maven build and tests
    inputs:
      mavenPomFile: 'pom.xml'
      goals: 'clean package'
      publishJUnitResults: true
      testResultsFiles: '**/surefire-reports/TEST-*.xml'
```

YAML Line	What it means
`trigger: - main`	Auto-trigger: every git push to the 'main' branch fires a new pipeline run
`pool: name: Default`	Use our self-hosted agent instead of a cloud agent

`checkout: self`	First step: download our repository code onto the agent machine
`task: Maven@3`	Use the official Azure DevOps Maven task (version 3) to run Maven
`goals: clean package`	Equivalent to running: mvn clean package in the terminal
`publishJUnitResults`	Upload test results to Azure DevOps so they appear in the Tests tab

5. Click Save and Run. Azure DevOps commits the YAML file to your repository and queues the first pipeline run.

PHASE 4 | First Pipeline Run – Grant Permission & Watch It Build

Step 5 – What happens after you click 'Save and Run'

After saving, Azure DevOps queues a run. You will be taken to the Runs list. This is what you will see:

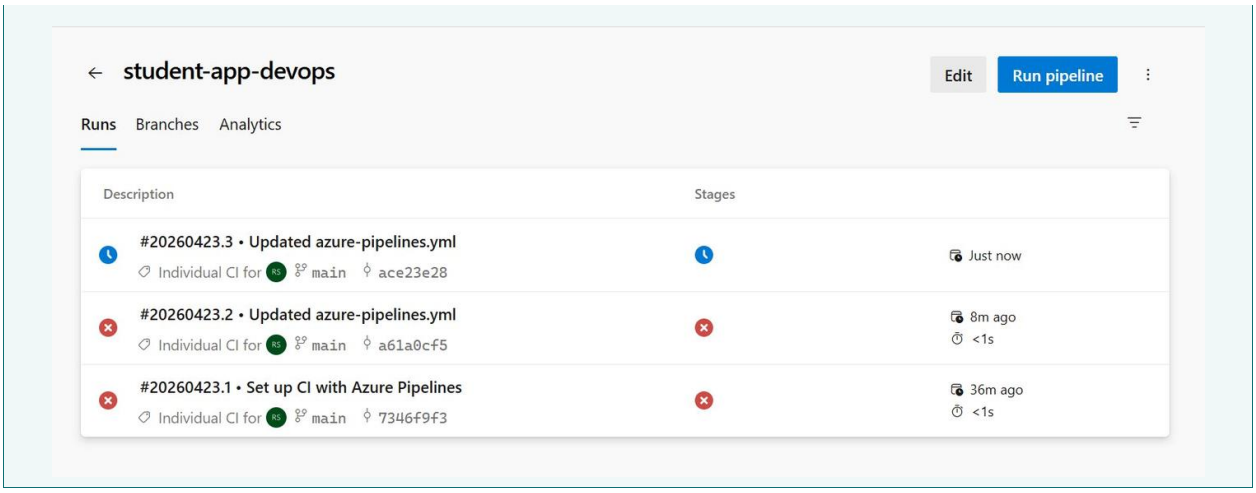

Figure 1: Pipeline Runs list – Runs #1 and #2 failed (red ✕) due to YAML issues. Run #3 is in progress (blue clock). This is normal – YAML is a learning process.

> ⚠ **Important:** Runs #1 and #2 showing red (failed) is completely normal when learning. A failure under <1 second means a YAML syntax error or wrong pool name. Fix the YAML and re-run. Run #3 succeeds after the YAML is corrected.

Step 6 – Grant Agent Pool Permission (One-Time Only)

On the very first successful pipeline run, Azure DevOps pauses and shows a security warning. This is a one-time safety check asking: 'Should this pipeline be allowed to use the Default agent pool?'

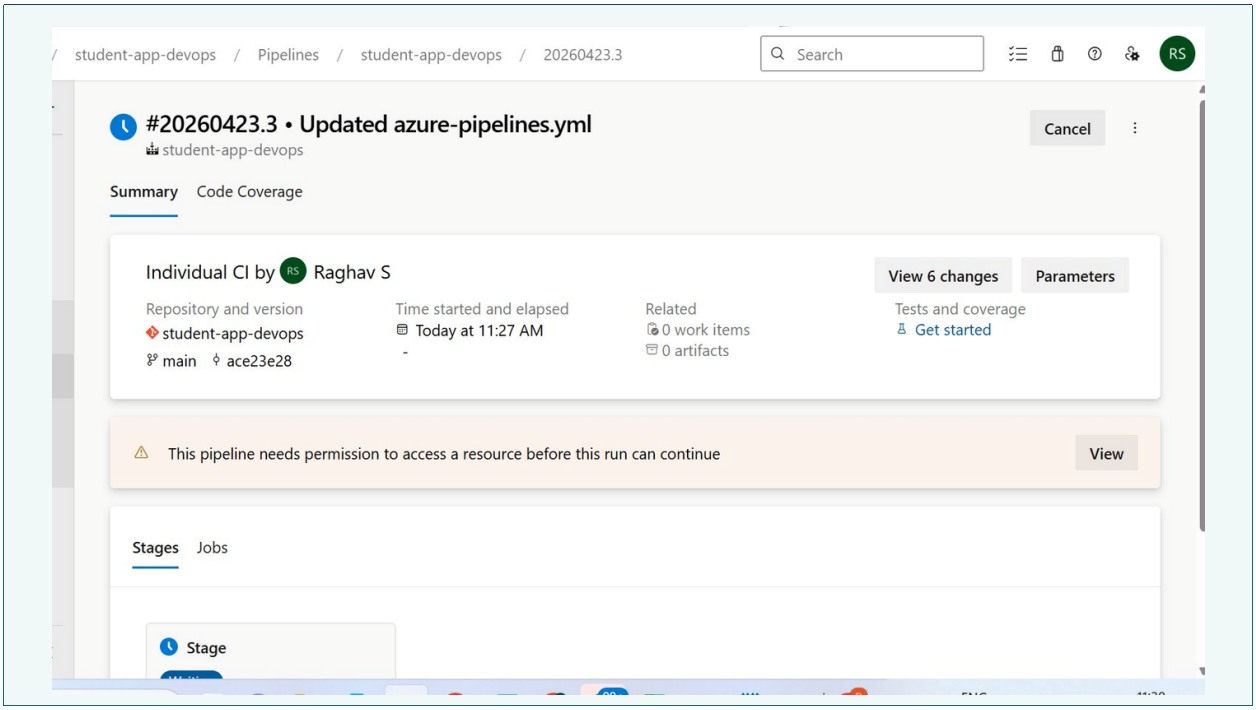

Figure 2: Pipeline paused with the orange warning banner 'This pipeline needs permission to access a resource before this run can continue'. Click the View button on the right.

1. Click the View button on the orange warning banner.

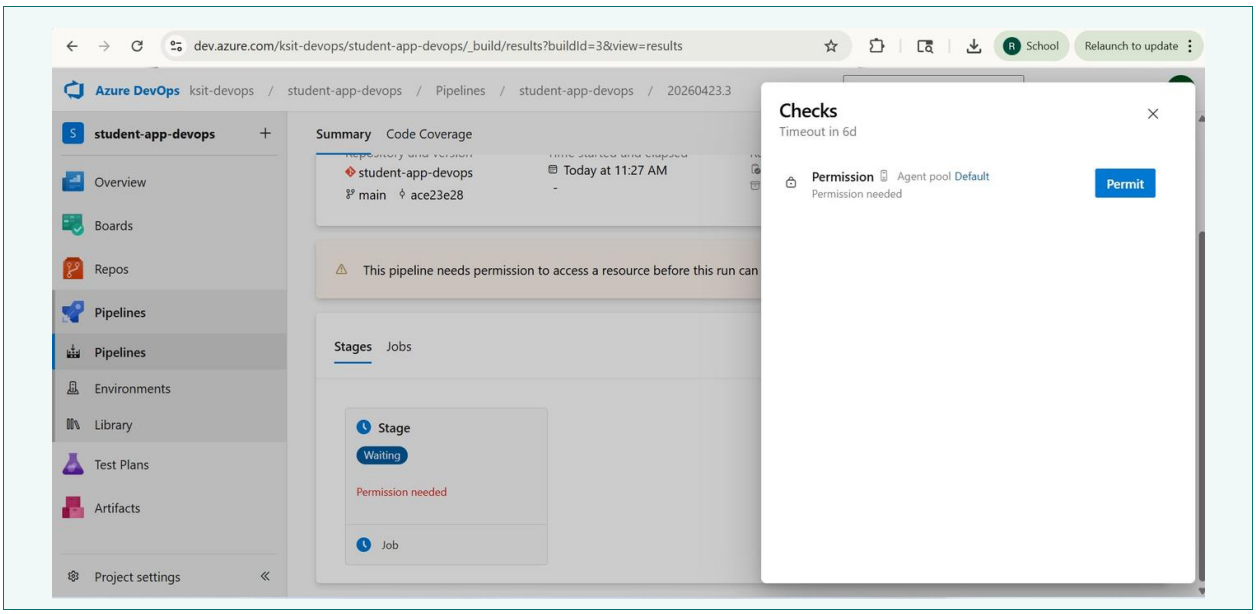

Figure 3: The Checks panel opens. You will see: Permission → Agent pool: Default → Permit button. Click Permit.

2. Click the blue Permit button. The pipeline will immediately resume.

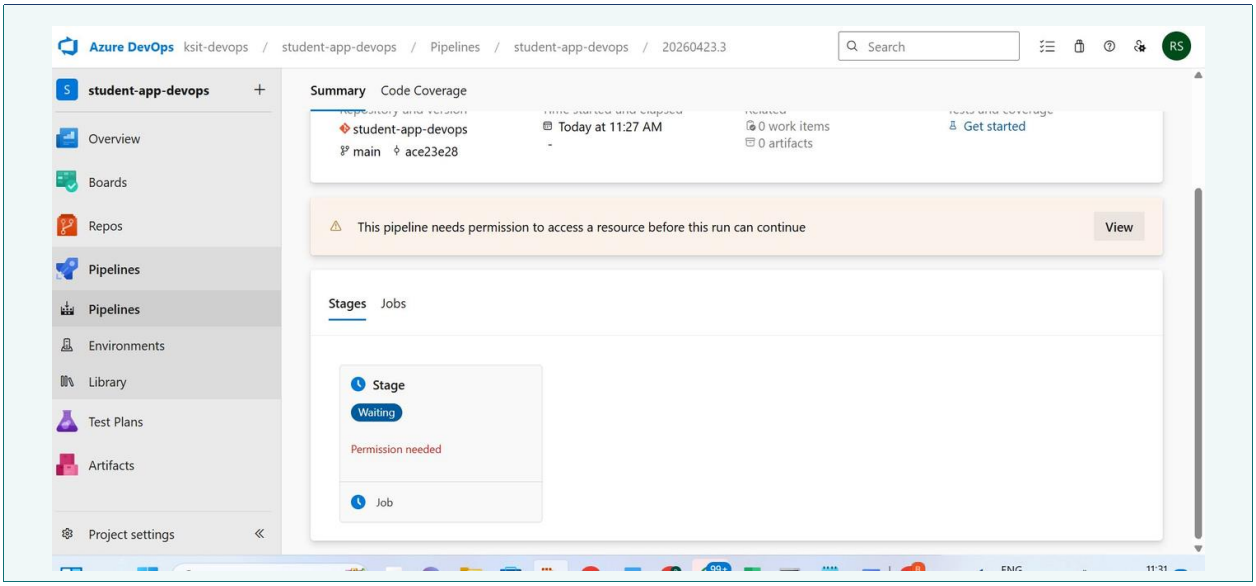

Figure 4: Stage is shown as 'Waiting – Permission needed'. After clicking Permit, it changes to Running within seconds.

> ⓘ **Note:** This permission is only needed ONCE per pipeline. All future runs will execute automatically without this prompt.

Step 7 – Monitor the Build in Real Time

After permission is granted, click on the running Job to open the live log view. You will see each step execute in real time:

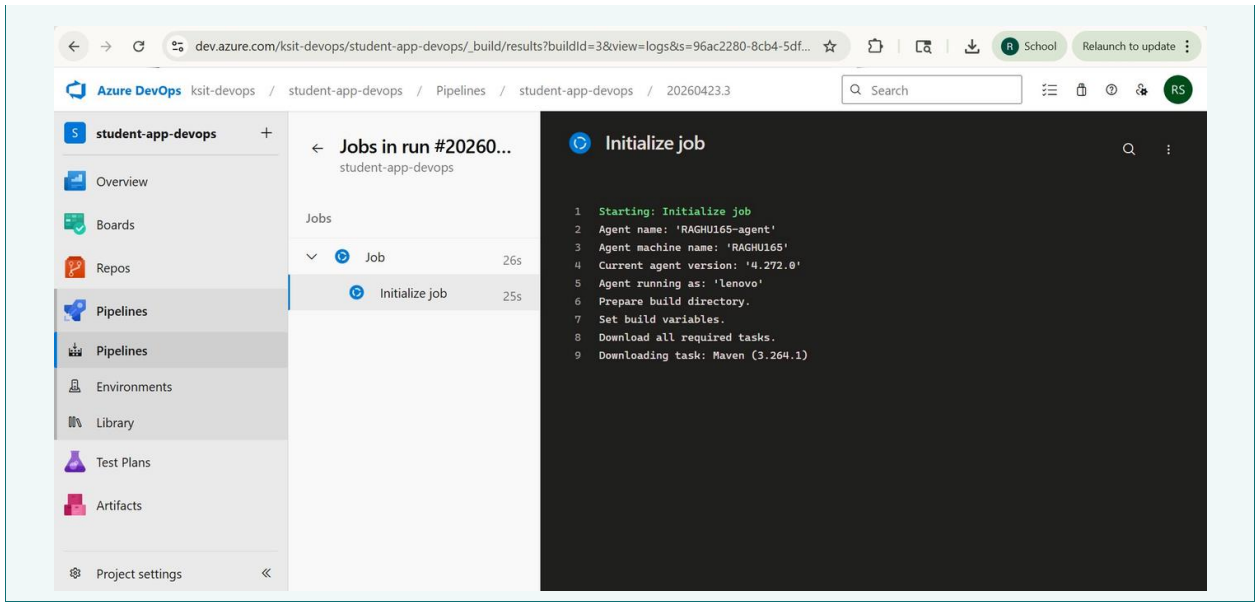

Figure 5: Initialize job step log. Notice: Agent name = RAGHU165-agent, Machine = RAGHU165 (your PC), Agent version = 4.272.0. This confirms YOUR machine is executing the build.

> ☑ **Expected Output:** Seeing your machine's name in the log (e.g., RAGHU165) is the key proof that your self-hosted agent is working correctly.

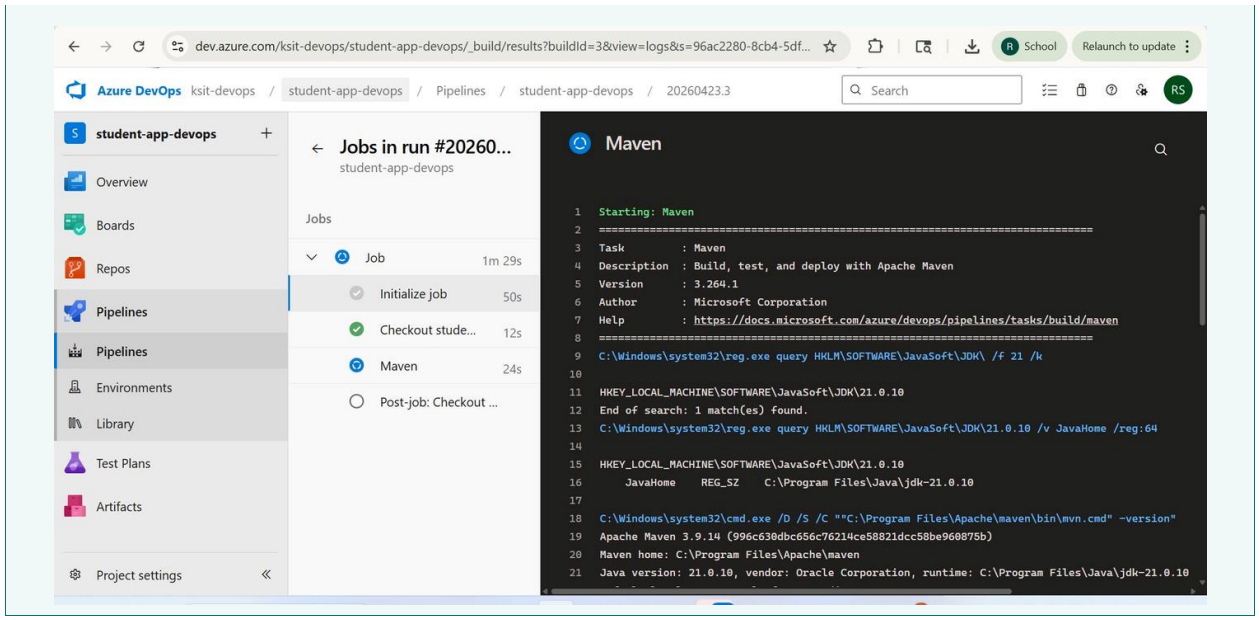

Figure 6: Maven step beginning – JDK 21 detected from Windows Registry (JavaHome = C:\Program Files\Java\jdk-21.0.10) and Maven 3.9.14 located. The build is about to start.

Step 8 – Verify BUILD SUCCESS

Scroll down in the Maven step log to see the test results and final outcome:

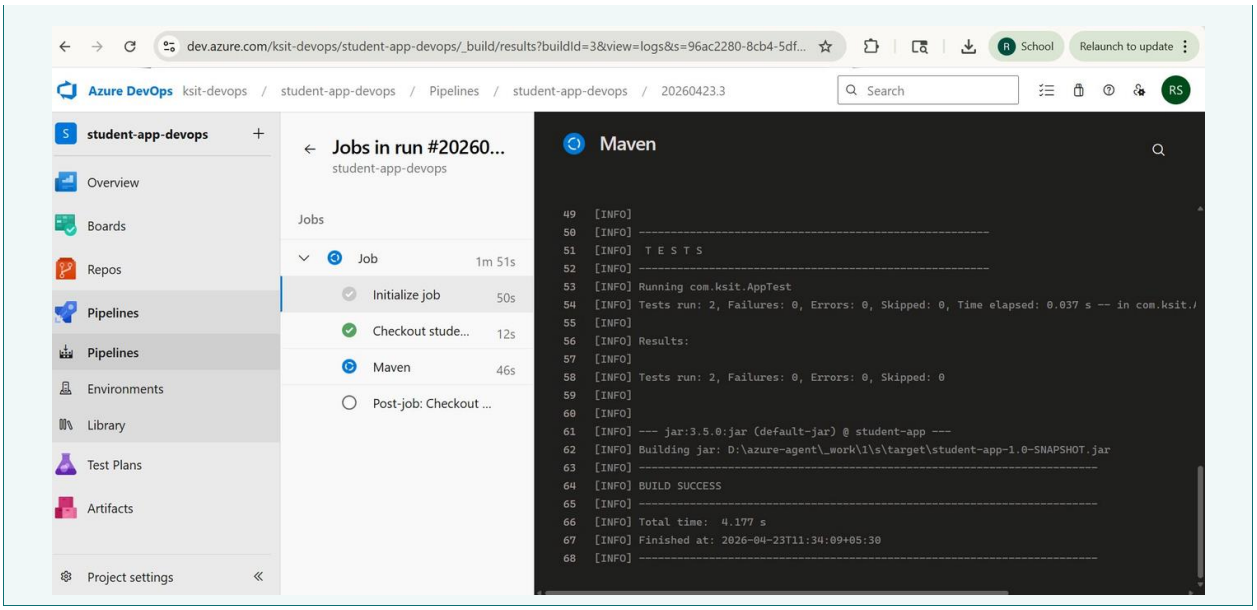

Figure 7: Maven build log showing Tests run: 2, Failures: 0, Errors: 0 → BUILD SUCCESS in 4.177 seconds. The JAR file student-app-1.0-SNAPSHOT.jar has been created.

```
[INFO] T E S T S
[INFO] Running com.ksit.AppTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO] Total time:  4.177 s
[INFO] Building jar: ...\target\student-app-1.0-SNAPSHOT.jar
```

☑ **Expected Output:** All 2 tests pass, 0 failures. BUILD SUCCESS. The pipeline run will now show a green checkmark on the Runs list page.

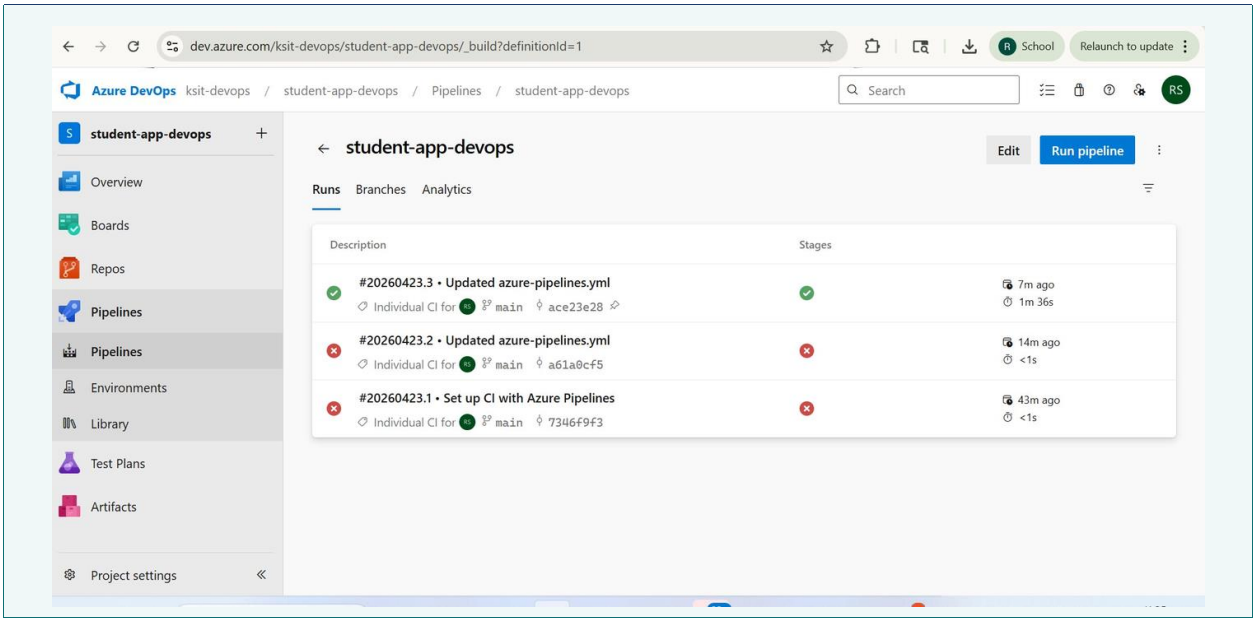

Figure 8: Pipeline Runs list – Run #3 shows a green ☑ checkmark confirming BUILD SUCCESS. Duration: 1 minute 36 seconds. This is your milestone screenshot for the record.

PHASE 5 | CI Auto-Trigger – Prove That CI is Working

Step 9 – Trigger a Build Automatically via Git Push

The defining feature of Continuous Integration is that builds trigger automatically without any manual action. You will now prove this:

1. Make a small change to your source code:

```
cd D:\Exercise\student-app
notepad src\main\java\com\ksit\App.java

# Change this line inside main():
#   System.out.println("Hello World!");
# To:
#   System.out.println("Hello Azure DevOps CI!");

# Save and close Notepad
```

2. Stage, commit, and push the change:

```
git add .
git commit -m "Test CI auto-trigger"
git push origin main

# If you get 'rejected: fetch first', run git pull first:
git pull
git push origin main
```

> ⚠ **Important:** If git push is rejected with 'fetch first': this means the remote repository has newer commits (e.g., the YAML file committed via browser). Just run git pull first to sync, then push again. This is normal and not an error.

3. Immediately open Azure DevOps → Pipelines in your browser. Do NOT click 'Run pipeline'. Just watch.

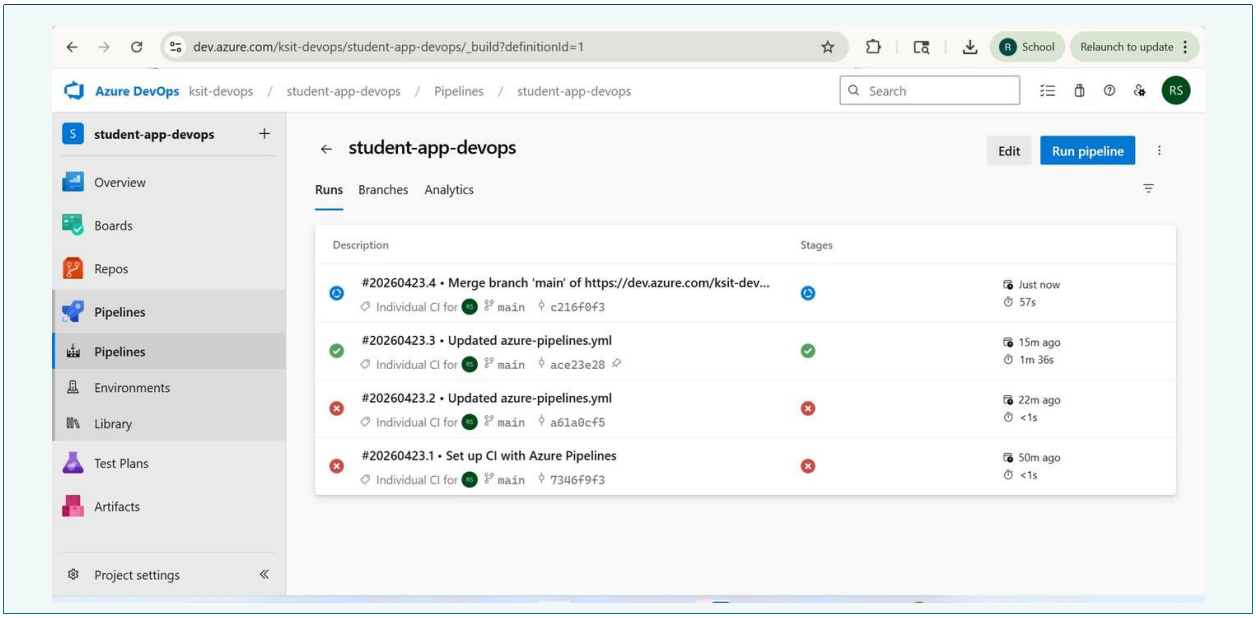

Figure 9: Run #4 auto-triggered! The description shows 'Merge branch main' with your commit message 'Test CI auto-trigger'. Azure DevOps detected the push and started a new run automatically – without you clicking anything.

☑ **Expected Output:** A NEW run (#4 or higher) appears automatically within 30–60 seconds of your git push. This is CI in action. The pipeline triggers itself based on code changes.

3. Common Errors and How to Fix Them

> ⓘ **Note:** Every error in this table was encountered during real lab execution. You are not the first person to see these – and each one has a simple fix.

Error / What you see	Why it happens	Exact Fix
Pipeline run fails in <1 second (no log output at all)	YAML syntax error or wrong indentation. Azure DevOps rejects the file before running anything.	Open Edit on the pipeline. Fix indentation – YAML uses 2 spaces, NOT tabs. Verify pool name is exactly: Default
'This pipeline needs permission to access a resource'	First-time security check for the agent pool – happens only once.	Click View button → Permit in the Checks panel. Done. Won't appear again for this pipeline.
Stage stuck at 'Waiting – Permission needed' even after Permit	Browser may not have refreshed the status.	Refresh the page. If still stuck, click the Cancel button and re-run the pipeline.
git push rejected: 'Updates were rejected, fetch first'	The Azure Repos remote has new commits (e.g., the YAML file) that your local copy doesn't have.	Run: git pull then: git push origin main
Agent shows Offline in Agent Pools page	The run.cmd process was closed or the machine was restarted.	Open PowerShell, cd D:\azure-agent, run .\run.cmd again. Keep the window open.
Maven build fails: 'JAVA_HOME not set' or 'mvn not found'	Environment variables missing from system PATH.	Add JAVA_HOME = C:\Program Files\Java\jdk-21.0.10 to System Environment Variables. Add %JAVA_HOME%\bin and C:\Program Files\Apache\maven\bin to PATH. Restart agent.
Run auto-triggered but build fails on Maven step	pom.xml missing or project folder structure incorrect.	Verify pom.xml exists in the root of the repository. Check: Repos → Files in Azure DevOps.
No new run appears after git push	trigger: section in YAML has wrong branch name.	Edit azure-pipelines.yml. Verify: trigger: - main (branch name must match exactly, case-sensitive)

4. Viva Questions & Model Answers

> ⓘ **Note:** These questions are based on what examiners typically ask for CI/CD programs. Read the model answers carefully – they show you what level of detail is expected.

Category A – Conceptual (Theory)

Q1 What is Continuous Integration (CI)?

Answer: CI is the DevOps practice of automatically building and testing code every time a developer pushes a change to the shared repository. The goal is to detect integration bugs as early as possible. In our lab, every git push to the main branch automatically triggered a Maven build and test run via Azure Pipelines.

Q2 What is the difference between a self-hosted agent and a Microsoft-hosted agent?

Answer: A Microsoft-hosted agent runs on a cloud VM managed by Microsoft – it starts fresh for every job. A self-hosted agent is a program installed on your own machine that runs jobs using your locally installed tools. We used a self-hosted agent (RAGHU165-agent) because our lab machine already had JDK 21 and Maven 3.9.14 installed.

Q3 What is azure-pipelines.yml and where is it stored?

Answer: It is a YAML configuration file that defines the entire CI pipeline – what triggers it, which agent to use, and what steps to run. It is stored INSIDE the Git repository along with the source code. This is called 'pipeline as code' – the build definition is version-controlled just like the application.

Q4 What does the trigger: section in the YAML do?

Answer: It specifies which branches should auto-trigger the pipeline when code is pushed. trigger: - main means: 'run this pipeline automatically every time someone pushes to the main branch'. Without this, the pipeline would only run if manually triggered.

Q5 What does mvn clean package do?

Answer: It runs four Maven phases in sequence: clean (deletes previous build output), compile (compiles .java files to .class), test (runs unit tests in src/test/), and package (packages compiled code into a JAR file). In our case it produced student-app-1.0-SNAPSHOT.jar.

Q6 Why did the pipeline need 'Permission to access a resource' on the first run?

Answer: Azure DevOps has a security policy that prevents a new pipeline from using shared resources like agent pools without explicit approval. This is to prevent a malicious pipeline from running

on production agents. Once an admin clicks Permit, the pipeline is trusted and future runs skip this check.

Q7 What is the difference between CI and CD?

Answer: CI (Continuous Integration) = automatically building and testing code on every commit. CD (Continuous Delivery) = automatically deploying the tested build to an environment (staging or production). In this lab we implemented CI only. A full CI/CD pipeline would add a Deploy stage after the Build stage succeeds.

Q8 What artifact was produced by this pipeline?

Answer: The Maven build produced student-app-1.0-SNAPSHOT.jar stored in the target/ directory on the agent machine at D:\azure-agent_work\1\s\target\student-app-1.0-SNAPSHOT.jar. In a real project, this JAR would be archived or deployed in the next stage.

Q9 What happened when the first two pipeline runs failed in under 1 second?

Answer: A failure in under 1 second means the pipeline did not even start executing. Azure DevOps rejected the YAML file itself before handing the job to the agent. This is typically caused by YAML syntax errors (wrong indentation, invalid key names, or wrong pool name). The fix was to correct the YAML and commit again.

Q10 If two developers push code at the same time, what happens?

Answer: Azure DevOps queues two separate pipeline runs. Each run is assigned to an available agent. If only one agent is registered, the second run waits in the queue until the first completes. Multiple agents in a pool can run jobs in parallel.

Category B – Practical / Hands-On

Q11 Show me the pipeline YAML you wrote. Explain each line.

Demonstrate: Point to the azure-pipelines.yml in Azure Repos and explain: trigger (auto-run on push to main), pool (use Default self-hosted agent), checkout (get the code), Maven@3 task (run mvn clean package), publishJUnitResults (show test results in the portal).

Q12 Where is the agent running? How do you know it is your machine?

Demonstrate: Open the Initialize job log of any successful run. Show lines: Agent name: RAGHU165-agent, Agent machine name: RAGHU165. These confirm the build ran on the local lab machine, not a cloud server.

Q13 Show me the BUILD SUCCESS message in the logs.

Demonstrate: Click on any green run → Job → Maven step. Scroll to the bottom of the log to show: [INFO] BUILD SUCCESS, Tests run: 2 Failures: 0, and the JAR file path.

Q14 How do you prove that CI auto-triggering is working?

Demonstrate: Show the Runs list. Point to Run #4 whose description says 'Test CI auto-trigger' (or your commit message). Explain that this run started automatically within 60 seconds of a git push, without clicking 'Run pipeline'.

Q15 What would you change in the YAML to also run the pipeline on a 'develop' branch?

Demonstrate: Modify the trigger section to: trigger: - main - develop This would make the pipeline fire on pushes to either the main or develop branch.

5. Lab Completion Checklist & Marks

#	Task to Complete	Marks	✓ Verified by Faculty
1	Azure DevOps project created and code pushed to Azure Repos (pom.xml visible in Repos → Files)	3	
2	Self-hosted agent registered and showing Online in Agent Pools → Default	3	
3	azure-pipelines.yml created with correct trigger, pool, and Maven task	3	
4	Pipeline run completes with BUILD SUCCESS and 2/2 tests passing (show green run)	4	
5	JAR artifact (student-app-1.0-SNAPSHOT.jar) confirmed in Maven step log	2	
6	CI auto-trigger demonstrated: git push caused a new run to start automatically (Run #4+)	5	

Total: 20 Marks	Marks Awarded: _____ / 20

Student Name & USN: _____ Date: _____ Faculty Signature: _____